

Battleships Logic Puzzle: Problem, Complexity, and Solutions

Introduction to Battleships Logic Puzzle

The Battleships Logic Puzzle is a single-player deduction game that challenges players to locate a hidden fleet of ships on a grid. The ships come in varying sizes, such as submarines occupying a single square or battleships spanning four squares. The grid is labeled with numeric clues indicating how many cells in each row or column are occupied by parts of ships.

The ships are placed either horizontally or vertically, and they do not touch each other, even diagonally. This rule creates additional constraints, requiring logical reasoning to deduce their positions. The puzzle begins with an empty grid and no direct information about ship placements. The player must use the clues and apply logical deductions to solve the puzzle.

For example, in a 5x5 grid with row clues [2, 3, 1, 0, 2] and column clues [2, 1, 3, 1, 1], the objective is to find a valid configuration of ship placements that satisfies these constraints.

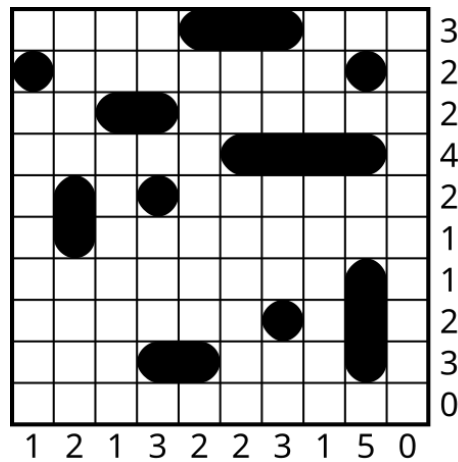


Figure 1: Solved Battleships Puzzle - Wikipedia

The NP-Completeness of the Battleships Puzzle

The Battleships Puzzle is classified as an NP-complete problem. This means that while verifying a solution (once provided) is computationally easy, finding the solution itself can be computationally infeasible for large grid sizes. The difficulty lies in the exponential growth of the solution space as the grid size increases and as the number of ships in the fleet grows.

To understand the NP-completeness of this puzzle, consider its reduction from other well-known NP-complete problems, such as SAT or Exact Cover. The constraints of the game—such as row and column clues, the specific fleet composition, and the no-adjacency rule—create a combinatorial explosion in the number of valid configurations. Verifying whether a given arrangement satisfies the clues and constraints is straightforward, but enumerating all possible configurations and checking each one is computationally expensive.

This classification has profound implications. It suggests that for sufficiently large grids or complex fleets, solving the puzzle within a reasonable time frame is impractical using brute force. Instead, heuristic methods, optimizations, and intelligent algorithms are necessary to efficiently solve the puzzle.

Algorithmic Approaches to Solving the Puzzle

Due to the computational complexity of the problem, various algorithmic approaches have been developed to solve Battleships puzzles. These approaches can be broadly categorized into three types: heuristic-based solvers, backtracking algorithms, and hybrid models.

Heuristic Solvers

Heuristic solvers mimic human reasoning by prioritizing moves based on logical deductions. For example, they may identify rows or columns where only one possible configuration exists based on the clues. These solvers are fast and work well for smaller grids but struggle with ambiguous or highly constrained puzzles.

Backtracking Algorithms

Backtracking, on the other hand, is a systematic search method that explores all possible ship placements recursively. It ensures that all constraints are satisfied at each step. While this approach guarantees a solution if one exists, it can be extremely slow for large grids due to its exhaustive nature.

Hybrid Models

Hybrid approaches combine the strengths of heuristics and backtracking. They use logical deductions to reduce the solution space before applying backtracking to handle remaining ambiguities. This balance allows for faster and more efficient solving while maintaining accuracy.

Implementation of the Game

My game implements the Battleships Logic Puzzle on grids of various sizes (5x5, 6x6, 7x7, and 10x10), staying true to the original rules with row and column clues guiding players to deduce the fleet's layout. Users can reveal hits or misses, flag water cells, or use algorithmic solvers for assistance.

One key difference from the standard puzzle is the absence of "ship tails and heads" logic, which traditionally allows players to follow a ship's trail once its orientation is partially revealed. While this could aid logical deduction, implementing it adds significant complexity to the game logic, requiring detailed handling of ship shapes and orientations.

	0	3	1	1	1	2	3	2	1	6
5	0.00	0.15	0.05	0.05	0.05	0.10	0.15	0.10	0.05	0.30
2	0.00	0.06	0.02	0.02	0.02	0.04	0.06	0.04	0.02	0.12
3	0.00	0.09	0.03	0.03	0.03	0.06	0.09	0.06	0.03	0.18
2	0.00	0.06	0.02	0.02	0.02	0.04	0.06	0.04	0.02	0.12
0	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
1	0.00	0.03	0.01	0.01	0.01	0.02	0.03	0.02	0.01	0.06
3	0.00	0.09	0.03	0.03	0.03	0.06	0.09	0.06	0.03	0.18
1	0.00	0.03	0.01	0.01	0.01	0.02	0.03	0.02	0.01	0.06
1	0.00	0.03	0.01	0.01	0.01	0.02	0.03	0.02	0.01	0.06
2	0.00	0.06	0.02	0.02	0.02	0.04	0.06	0.04	0.02	0.12

Figure 2: Own Implementation - Probability Map Over Game Grid

I deliberately avoided this feature to keep the focus on algorithms, as this project emphasizes solving methods rather than game design. This simplification does not detract from the challenge of the puzzle and allows us to test and evaluate algorithms in a streamlined environment.

Enhanced Probability Map with Logical Deductions

The enhanced probability map is central to my first algorithmic approach. It combines a naive probability distribution with logical deduction techniques to improve accuracy and efficiency when solving the puzzle. Probabilities are computed based on how many ship parts are still required in each row and column, relative to the number of unrevealed cells.

Mathematical Formula

For a cell (r, c) , the probability of containing a ship part is computed as:

$$P(r, c) = \frac{\text{row_clues}[r] - \text{hits_in_row}[r]}{\text{hidden_in_row}[r]} + \frac{\text{col_clues}[c] - \text{hits_in_col}[c]}{\text{hidden_in_col}[c]}$$

Where:

- `row_clues[r]` and `col_clues[c]` are the number of ship parts required in row r and column c , respectively.
- `hits_in_row[r]` and `hits_in_col[c]` are the number of ship parts already found (hits) in row r and column c .
- `hidden_in_row[r]` and `hidden_in_col[c]` are the number of unrevealed (unflagged) cells remaining in row r and column c .

This formula ensures that cells in rows or columns with fewer remaining required ships and fewer hidden cells are assigned higher probabilities, focusing attention where ships are most likely to be found.

Logical Deductions

The algorithm supplements this probability map with logical rules:

1. Full Rows or Columns:

- If a row or column has exactly as many unrevealed cells as remaining required ship parts, all such cells are marked as ships.

2. Empty Rows or Columns:

- If a row or column's ship count is satisfied, all remaining cells in that row or column are flagged as water.

3. Isolated Cells:

- If only one cell in a row or column can satisfy the remaining clues, it is immediately marked as a ship part.

The probability-based solution enhanced with logical deduction achieves an accuracy of 62.5% based on simulated games, which is respectable but highlights some limitations. Its primary drawback is the lack of deeper rational reasoning and contextual awareness. For instance, if two hit cells flank an unrevealed cell and a ship of length 3 is still unplaced, logic strongly suggests the unrevealed cell is part of that ship. However, the algorithm assigns probabilities independently, missing such contextual cues. This limitation can lead to suboptimal decisions, such as prioritizing cells with marginally higher probabilities while ignoring obvious patterns dictated by the remaining fleet. Addressing these gaps requires a more sophisticated approach that considers ship configurations and board constraints holistically.

Backtracking Algorithm

The backtracking algorithm is a systematic approach to solving the Battleships puzzle by exploring all possible ship placements on the board that satisfy the constraints. The method ensures that each configuration adheres to the given row and column clues, the fleet size, and the no-adjacency rule. By recursively placing ships and validating the configuration, the algorithm identifies all valid solutions to the puzzle.

How the Algorithm Works

1. Initialization:

- The board is initialized with zeros (indicating empty cells).
- Known hits and misses are applied to the board based on the game's current state.
- Row and column constraints are adjusted to account for these known values.

2. Recursive Backtracking:

- The algorithm places ships one at a time, starting with the largest ship.
- It attempts to place each ship in every possible valid position and orientation (horizontal or vertical) on the board.
- If a placement is invalid (e.g., violating adjacency or constraints), the algorithm backtracks and tries a different configuration.

3. Validation: Each ship placement is validated using the constraints:

- The row and column must have enough remaining capacity to accommodate the ship.
- The ship must not overlap with existing ships or violate the no-adjacency rule.

- The placement must align with the known hits and misses.

4. Completion Check:

- When all ships are successfully placed and the row and column constraints are fully satisfied, the current configuration is saved as a valid solution.

5. Backtracking:

- If a placement leads to a conflict or if all possibilities are exhausted, the algorithm removes the last placed ship and tries a different position or orientation for it.

Backtracking Algorithm Performance

The backtracking algorithm is a brute-force method that systematically explores all possible configurations of the board to solve the puzzle. While it is exhaustive in its approach and guarantees that all valid solutions are explored, its performance and behavior can vary significantly depending on the size of the grid and the constraints.

Performance on Larger Grids

The complexity of the algorithm grows exponentially with the size of the grid and the number of ships. For grids larger than 8×8 , the number of possible configurations becomes astronomically large, making it impractical to run within a reasonable time frame. This is why backtracking is generally unsuitable for larger boards without optimizations or heuristics to reduce the search space.

Solving Part of the Puzzle

One of the key observations is that the algorithm often solves part of the puzzle with 100% accuracy but then gets stuck, claiming there are no more possible solutions. This behavior can happen for several reasons:

1. Constraint Satisfaction Exhaustion:

- The backtracking algorithm relies heavily on the constraints provided by the row and column clues, along with the known hits and misses.
- If the known clues and hits/misses are insufficient to uniquely determine the rest of the puzzle, the algorithm might legitimately conclude that no more forced placements are possible.

2. Ambiguous Puzzle States:

- There are cases where the remaining parts of the puzzle have multiple valid solutions, and the algorithm cannot deduce a unique placement without further information.

- For example, if a row has two empty cells and the clue allows one ship, either cell could potentially hold the ship. The algorithm cannot choose between these equally valid options.

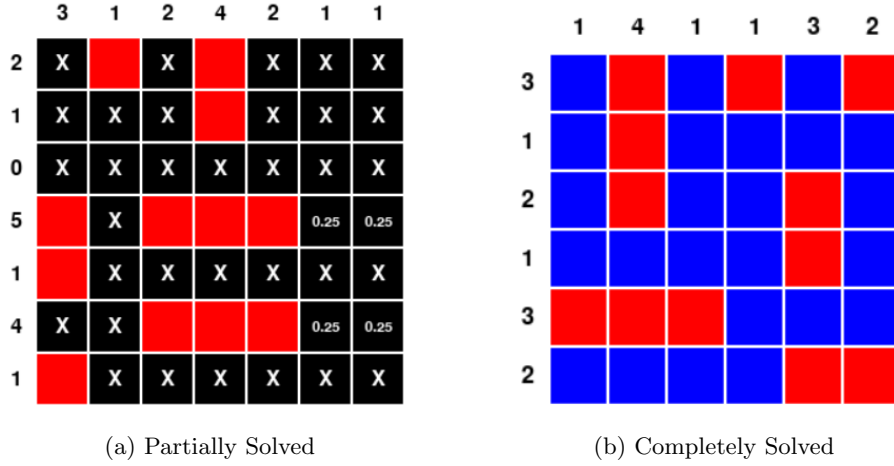


Figure 3: Games solved by the Backtracking Algorithm

Is This Behavior Normal?

If the algorithm gets stuck due to insufficient constraints or ambiguous states where multiple solutions exist, this is normal and expected. It simply reflects the limitations of the deterministic nature of backtracking in solving puzzles without complete information. In such cases, human intuition or additional logical deductions may be necessary to progress further.

Improving Puzzle Solvability

Removing single-cell ships from the armada significantly improves the solvability of Battleship puzzles, particularly for backtracking algorithms. Single-cell ships create high ambiguity because they can fit into any unrevealed square, drastically increasing the number of potential configurations. This complicates logical deductions and introduces a large branching factor in the search tree, making the algorithm slower and more prone to concluding that no solutions exist.

By focusing on larger ships, puzzles become more structured, with clearer constraints and fewer isolated cells. Larger ships provide stronger dependencies across multiple cells, allowing logical deductions to work more effectively. Additionally, reducing the risk of contradictions from adjacency constraints simplifies the solving process, enabling backtracking algorithms to solve puzzles faster and

with greater consistency. Removing single-cell ships thus strikes a balance between challenge and solvability.

Conclusion

The Battleships Logic Puzzle provides a fascinating challenge in computational complexity and algorithm design. The NP-completeness of the puzzle underscores the difficulty of solving it for large grids, necessitating the use of heuristic methods, backtracking, and hybrid approaches.

By combining probability maps, logical deductions, and systematic search methods, solvers can efficiently tackle puzzles while navigating their inherent ambiguities. Simplifying the fleet composition by removing single-cell ships further enhances solvability, making it possible to achieve better results within reasonable time frames. The insights gained from solving these puzzles have broader implications for logic-based problem-solving and computational strategies.

The Forgotten Graphs

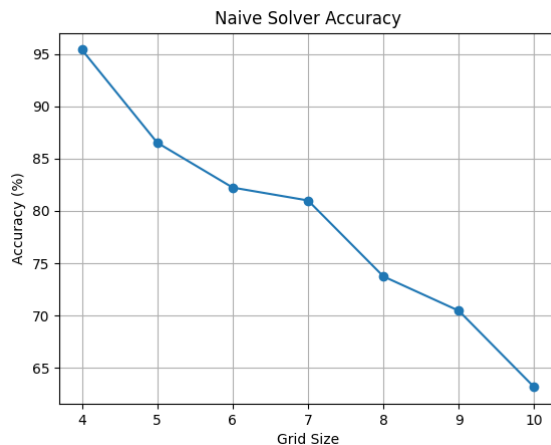


Figure 4: Naive Solver Accuracy

Naive Solver Accuracy

The first graph shows the accuracy of the naive solver across grid sizes ranging from 4×4 to 10×10 . The accuracy declines consistently as the grid size increases. For a 4×4 grid, the solver achieves an accuracy of approximately 95%, whereas for a 10×10 grid, the accuracy drops to around 65%. This

reduction is expected as the complexity and number of possibilities increase with larger grid sizes, making the naive approach less effective in identifying ship placements.

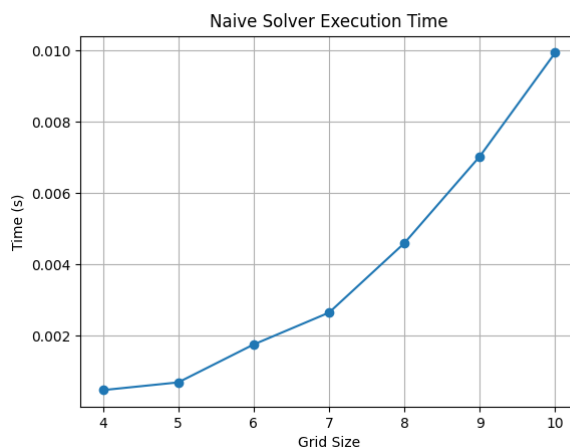


Figure 5: Naive Solver Execution Time

Naive Solver Execution Time

The second graph highlights the execution time of the naive solver across the same grid sizes. The time increases steadily with grid size, showing a roughly linear trend. For a 4×4 grid, the average execution time is around 0.001 seconds, while for a 10×10 grid, it rises to approximately 0.01 seconds. This increase reflects the higher computational effort required to calculate probabilities and make decisions as the board size grows.

Backtracking Solver Execution Time

The third graph presents the execution time of the backtracking solver for grid sizes ranging from 4×4 to 9×9 . Testing for the 10×10 grid was excluded because the execution time would be prohibitively high, potentially skewing the graph and undermining the other data points. The graph shows an exponential growth in execution time as the grid size increases. For example, while the execution time for a 4×4 grid is negligible, it becomes significant for a 9×9 grid, exceeding 0.35 seconds on average. This behavior is expected due to the exponential nature of backtracking algorithms as

This first graphs displays the execution time of the backtracking algorithm which

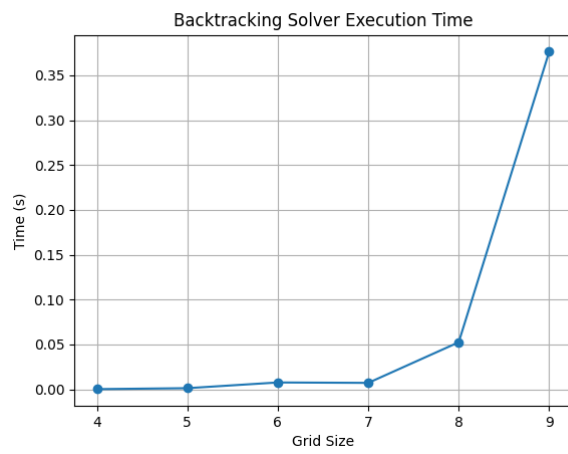


Figure 6: Backtracking Execution Time